

CUSTOMER CHURN ANALYSIS

An end-to-end SQL → Python analytics workflow that traces subscription customers from raw relational tables through cleaning, feature engineering, KPI modelling and visualization — ending in a set of retention-focused business insights.

RISHABH SINGH

TOOLS & LIBRARIES

Pandas

NumPy

SQLite3

Matplotlib

Seaborn

ANALYTICAL WORKFLOW

- 01 SQL Connect
- 02 Data Importing in Jupyter Notebook
- 03 Exploratory Data Analysis
- 04 Data Cleaning
- 05 Analysis (KPI)
- 06 Visualisations
- 07 Insights

Importing Libraries & Source Data

The notebook opens a SQLite database of three related tables — customer profile, subscription, and support — and loads each into its own Pandas DataFrame for exploration.

Importing Libraries

In [1]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

Creating Dataframes

In [2]

```
conn = sqlite3.connect('customer_churn.db')
sql_query = '''
    SELECT name
    FROM sqlite_master
    WHERE type = 'table'
'''
tables = pd.read_sql(sql_query, conn)

for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created Dataframe: df_{table_name}")

conn.close()
```

OUT

```
Created Dataframe: df_db_customer
Created Dataframe: df_db_subscription
Created Dataframe: df_db_support
```

Connect SQL to Python

In [3]

```
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

conn.close()
```

OUT

```
Table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date',
'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason',
'monthly_charges', 'cltv', 'churn_score']

Table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

Exploratory Data Analysis

A first look at each raw table before any cleaning is applied.

df_db_customer.head()

In [5]

top 5 rows
df_db_customer.head()

customerid	name	country	state	gender	dob	interests	pincode
0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23	None	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30	None	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05	drama	None

df_db_subscription.head()

In [6]

df_db_subscription.head()

customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date
0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	—
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	—
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	—
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28

df_db_support.head()

In [7]

df_db_support.head()

customerid	complaint_date	escalations	csat_score	comment
0003-MKNFE	2024-08-28	N	60	service issue
0003-MKNFE	2024-08-28	Y	10	demanded refund
0013-EXCHZ	2024-01-20	Y	20	—
0013-MHZWF	2025-03-18	N	90	guidance to renew
0013-SMEOE	2024-11-01	N	30	—

Structural Check — df_db_customer.info()

In [8]

```
df_db_customer.info()
```

OUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0  customerid  21 non-null    object
 1  name        21 non-null    object
 2  country     18 non-null    object
 3  state       21 non-null    object
 4  gender      21 non-null    object
 5  dob         21 non-null    object
 6  interests   4 non-null     object
 7  pincode     0 non-null     object
dtypes: object(8)
memory usage: 1.4+ KB
```

3 missing country values, only 4/21 customers have an interests tag, and pincode is entirely empty — each is addressed in Data Cleaning below.

Data Cleaning — Customer Table

a) Rename column — name → customer_name

```
In [9]

df_db_customer.rename(columns={'cust_name': 'customer_name'}, inplace=True)
df_db_customer.tail()
```

b) Drop unused columns — interests, pincode

```
In [10]

df_db_customer.drop(columns=['interests', 'pincode'])
```

id	customerid	name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05
5	0013-MHZWF	durga	None	Delhi	Women	1988-12-10
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14
8	0015-UOCOJ	maya	None	Kathmandu	Women	1985-07-07
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18
12	0018-NYROU	chitra	None	Telangana	Female	2004-12-01
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06

Data Cleaning — Customer Table (cont.)

c) Fix dtype — dob → datetime

In [11]

```
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

d) Standardize categories — gender

In [12]

```
df_db_customer['gender'].replace({'Men': 'Male', 'Women': 'Female'})
```

OUT

```
0    Male
1    Male
2   Female
3    Male
4   Female
5   Female
6   Female
7    Male
8   Female
9    Male
10   Male
11  Female
12  Female
13  Female
14    Male
15  Female
16  Female
17  Female
18    Male
19    Male
20  Female
Name: gender, dtype: object
```

In [13]

```
df_db_customer['gender'].unique()
```

OUT

```
array(['Male', 'Female', 'Women', 'Men'], dtype=object)
```

The `.replace()` call above wasn't assigned back to the column, so Men / Women still appear afterward — a good example of why re-checking `.unique()` after a transform matters.

Data Cleaning — Customer Table (cont.)

e) Fix missing values — country

In [14]

```
df_db_customer['country'].isna()
```

OUT

```
0    False
1    False
2    False
3    False
4    False
5     True
6    False
7    False
8     True
9    False
10   False
11   False
12     True
13   False
14   False
15   False
16   False
17   False
18   False
19   False
20   False
Name: country, dtype: bool
```

In [15]

```
df_db_customer[df_db_customer['country'].isna()]
```

id	customerid	name	country	state
5	0013-MHZWF	durga	None	Delhi
8	0015-UOCOJ	maya	None	Kathmandu
12	0018-NYROU	chitra	None	Telangana

In [16]

```
state_country_mapping = (df_db_customer.dropna(subset=['country'])
                          .set_index('state')['country'].to_dict())

df_db_customer['country'] = df_db_customer['country'].fillna(
    df_db_customer['state'].map(state_country_mapping)
)
```

In [17]

```
df_db_customer[df_db_customer['country'].isna()]
```

OUT

```
Empty DataFrame
Columns: [customerid, name, country, state, gender, dob, interests, zipcode]
Index: []
```

All 3 missing country values were recovered by mapping each customer's state to a known country from the rest of the table — no rows dropped.

Data Cleaning — Subscription & Support Tables

Subscription — dtype check & date parsing

In [19]

```
df_db_subscription.info()
```

OUT

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customerid            21 non-null    object
1   subscription_start_date 21 non-null    object
2   subscription_type      21 non-null    object
3   renewal_date          21 non-null    object
4   plan_type              21 non-null    object
5   contract_type          21 non-null    object
6   cancellation_date      6 non-null     object
7   cancellation_reason     6 non-null     object
8   monthly_charges        21 non-null    float64
9   cltv                   21 non-null    int64
10  churn_score             21 non-null    int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB
```

In [20]

```
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']
df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

OUT

```
dtypes: datetime64[ns](3), float64(1), int64(2), object(5)
memory usage: 1.9+ KB
```


Data Cleaning — Subscription & Support Tables (cont.)

Support — drop empty / free-text columns

In [21]

```
df_db_support.info()
```

```
OUT
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   customerid      9 non-null     object
1   complaint_date  9 non-null     object
2   escalations     9 non-null     object
3   csat_score      9 non-null     int64
4   col_1           0 non-null     object
5   comment         4 non-null     object
dtypes: int64(1), object(5)
memory usage: 564.0+ bytes
```

In [22]

```
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

In [23]

```
df_db_support.info()
```

```
OUT
Data columns (total 4 columns):
 customerid, complaint_date, escalations, csat_score
dtypes: int64(1), object(3)
memory usage: 420.0+ bytes
```

Feature Engineering & Merging

A churn flag is derived, tables are joined on customerid, and a duplicate-row issue surfaces and gets resolved.

Churn flag from cancellation_date

In [24]

```
df_db_subscription['churn_flag'] = np.where(
    df_db_subscription['cancellation_date'].notna(), 1, 0
)
```

First merge — reveals a duplicate-row problem

In [29]

```
(df_db_subscription.merge(df_db_customer, on='customerid', how='left')
 .merge(df_db_support, on='customerid', how='left'))
```

In [30]

```
# Having duplicate problem.
df_db_subscription.shape
```

```
OUT
(21, 12)
```

In [31]

```
df_db_subscription['customerid'].nunique()
```

```
OUT
21
```

In [32]

```
df_db_support['customerid'].nunique() # problem
#df_db_support['customerid'].size
```

```
OUT
7
```

21 unique subscriptions merged against a support table with 7 unique customers but **9 rows** — the merge fans out to 23 rows. Root cause: repeat complaints per customer.

Feature Engineering & Merging (cont.)

Diagnosing & resolving the duplicates

```
In [36]

df_db_support.groupby('customerid')['customerid'].transform('count')

OUT
0 2  1 2  2 1  3 1  4 1  5 1  6 1  7 2  8 2
Name: customerid, dtype: int64
```

```
In [37]

df_db_support['complaint_count'] = (
    df_db_support.groupby('customerid')['customerid'].transform('count')
)
```

customerid	complaint_date	escalations	csat_score	complaint_count
0003-MKNFE	2024-08-28	N	60	2
0003-MKNFE	2024-08-28	Y	10	2
0013-EXCHZ	2024-01-20	Y	20	1
0013-MHZWF	2025-03-18	N	90	1
0013-SMEOE	2024-11-01	N	30	1
0017-IUDMW	2024-04-10	Y	25	1
0019-EFAEP	2024-09-27	Y	30	1
0022-TCJCI	2024-09-13	Y	10	2
0022-TCJCI	2024-09-14	N	90	2

```
In [39]

df_db_support = (df_db_support
    .sort_values('complaint_date')
    .drop_duplicates('customerid', keep='last'))
```

Keeping the *most recent* complaint per customer collapses support down to one row each — 9 rows → 7 — without losing the latest signal.

Feature Engineering & Merging (cont.)

Final merge & export

In [41]

```
df = (df_db_subscription
      .merge(df_db_customer, on='customerid', how='left')
      .merge(df_db_support, on='customerid', how='left'))
```

In [42]

```
df.shape
```

OUT

```
(21, 21)
```

In [43]

```
df.to_csv('exported_churn_data.csv', index=False)
```

Resulting schema

In [44]

```
df.columns
```

OUT

```
Index(['customerid', 'subscription_start_date', 'subscription_type',
      'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
      'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
      'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
      'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='object')
```

Data Analysis — KPI Dashboard

Ten headline metrics computed directly from the merged, cleaned dataset (21 customers).

28.57% CHURN RATE 6 of 21 customers	71.43% RETENTION RATE 100 – churn rate	Rs 18.85 ARPU avg. monthly_charges
Rs 73.94K REVENUE AT RISK sum, churned users	1,520 days AVG. TENURE ≈ 4.2 years	19.05% ESCALATION RATE share flagged 'Y'

In [46]

```
churn_rate = df['churn_flag'].mean() * 100
print("Churn rate =", round(churn_rate, 2), "%")
```

OUT

Churn rate = 28.57 %

In [48]

```
retention_rate = 100 - churn_rate
print("Retention Rate =", round(retention_rate, 2), "%")
```

OUT

Retention Rate = 71.43 %

KPI Dashboard — Churn by Segment

Churn by plan type

```
In [51]

churn_by_plan = (df.groupby('plan_type')['churn_flag']
                 .mean().mul(100).round(2)
                 .reset_index(name='churn_rate_pct'))
```

plan_type	churn_rate_pct
Basic	60.00
Premium	14.29
Standard	22.22

Churn by state

```
In [53]

churn_by_state = (df.groupby('state')
                  .agg(churn_rate_pct=('churn_flag', lambda x: x.mean()*100),
                      total_revenue=('monthly_charges', 'sum'),
                      user_count=('customerid', 'nunique'))
                  .round(2).reset_index())
```

state	churn_rate_pct	total_revenue	user_count
Delhi	25.00	52.96	4
Karnataka	100.00	20.98	2
Kathmandu	0.00	20.98	2
Maharashtra	0.00	50.97	3
Meghalaya	66.67	42.97	3
Nagaland	0.00	22.99	1
Rajasthan	0.00	36.98	2
Telangana	50.00	30.98	2
Uttar Pradesh	0.00	115.98	2

KPI Dashboard — Acquisition, Revenue & Risk

Churn by acquisition channel (subscription_type)

```
In [55]

churn_by_subscription = (df.groupby('subscription_type')
    .agg(churn_rate_pct=('churn_flag', lambda x: x.mean()*100),
        total_revenue=('monthly_charges', 'sum'),
        user_count=('customerid', 'nunique'))
    .round(2).reset_index())
```

subscription_type	churn_rate_pct	total_revenue	user_count
Organic	0.00	145.91	9
Paid	16.67	174.94	6
Refferal	83.33	74.94	6

KPI Dashboard — Acquisition, Revenue & Risk (cont.)

Revenue, tenure, escalation & complaint metrics

In [57]

```
arpu = df['monthly_charges'].mean()
print('ARPU =', round(arpu, 2))
```

OUT

ARPU = 18.85

In [59]

```
today = pd.Timestamp.today()
df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),
    (df['cancellation_date'] - df['subscription_start_date']).dt.days,
    (today - df['subscription_start_date']).dt.days
)
avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (days) =", round(avg_tenure))
```

OUT

Avg Tenure (days) = 1520

In [61]

```
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at risk =", revenue_at_risk, "K")
```

OUT

Revenue at risk = 73.94 K

In [63]

```
esclation_rate = (df['escalations']=='Y').mean() * 100
print("Escalation rate =", round(esclation_rate, 2), "%")
```

OUT

Escalation rate = 19.05 %

In [65]

```
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Complaints per user =", round(avg_complaints, 2))
```

OUT

Avg Complaints per user = 0.43

Encoding & Correlation

Categorical fields are converted to numeric form so churn can be correlated against support behaviour, and a risk tier is engineered from churn_score.

Encoding escalations (Y/N → 1/0)

```
In [74]

df['escalations'] = (
    df['escalations'].astype('string').str.strip().str.upper()
    .map({'Y': 1, 'N': 0})
)
```

```
In [76]

df['escalations'] = df['escalations'].fillna(0)
```

Correlation — escalations vs. churn

```
In [77]

corr_df = df[['escalations', 'churn_flag']].dropna()
correlation = corr_df['escalations'].corr(corr_df['churn_flag'])
print("Correlation between escalations vs churn =", round(correlation, 2))

OUT
Correlation between escalations vs churn = 0.77
```

A 0.77 correlation is a strong practical signal: escalated support tickets are one of the clearest early-warning flags for churn in this dataset.

Deriving churn_risk from churn_score

```
In [78]

conditions = [
    (df['churn_score'] < 50),
    (df['churn_score'] >= 50) & (df['churn_score'] < 70),
    (df['churn_score'] >= 70)
]
choices = ['low', 'med', 'high']
df['churn_risk'] = np.select(conditions, choices, default='unknown')
```

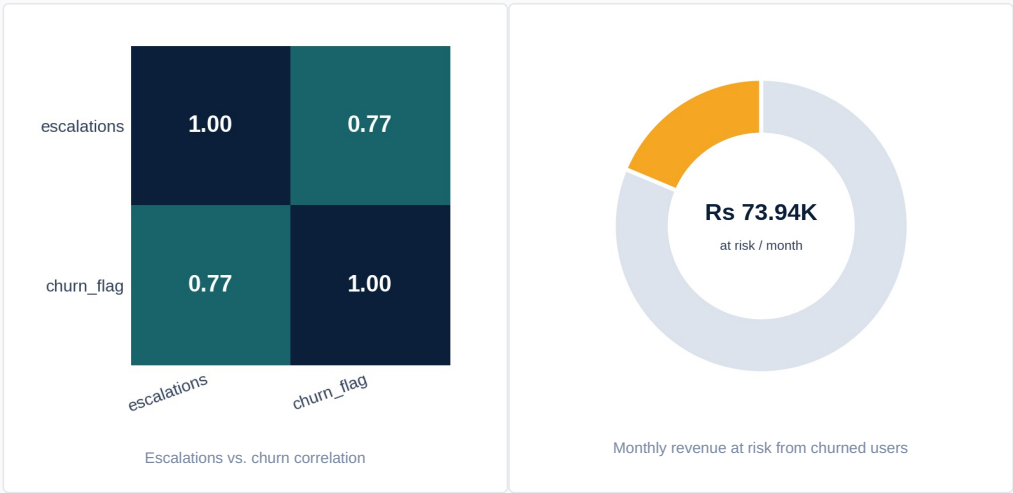
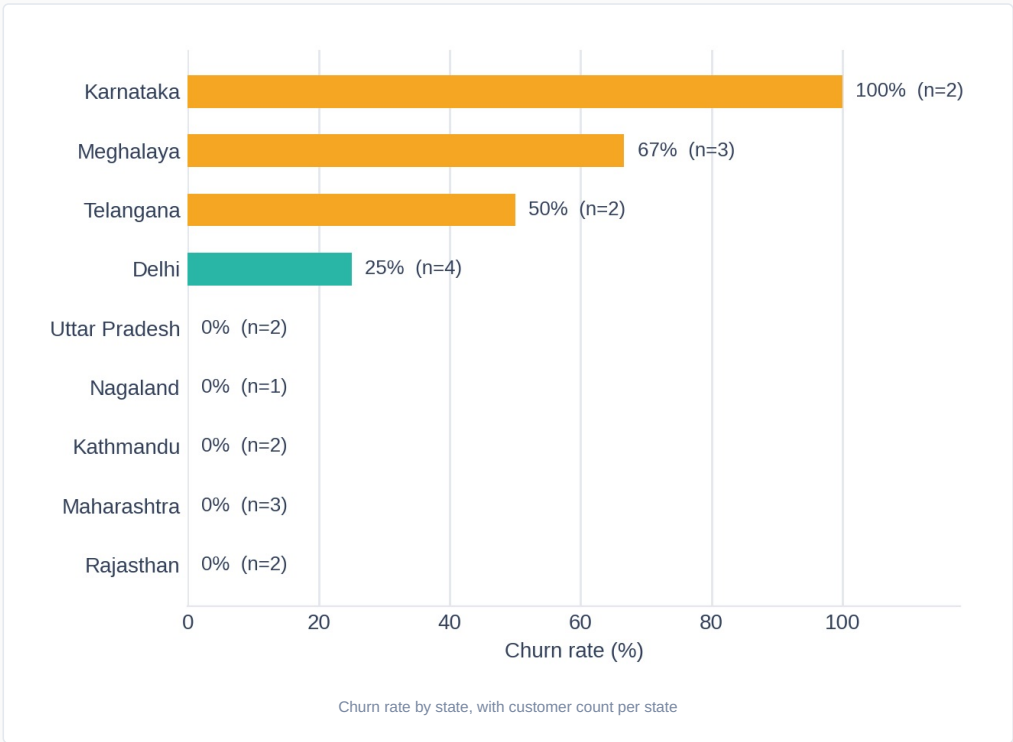
customerid	churn_score	churn_risk
0020-JDNXP	62	med
0021-IKXGC	27	low
0022-TCJCI	99	high
0023-HGHWL	7	low
0023-UYUPN	47	low

Visualisations

Key relationships from the KPI stage, charted for quick scanning.



Visualisations (cont.)



Pivot Table Summary

```
In [97]

pd.pivot_table(
    df,
    values='churn_flag',
    index='plan_type',
    aggfunc='mean'
)
```

plan_type	churn_flag (mean)
Basic	0.600000
Premium	0.142857
Standard	0.222222

This mirrors the earlier groupby result and confirms Basic-plan customers churn at roughly 4× the Premium rate.

Key Insights

Six findings from the full workflow, ranked by business impact.

01 — RETENTION CHALLENGE

Nearly 1 in 3 customers has left

A 28.57% churn rate leaves significant room for retention improvement across the base.

02 — BASIC PLAN RISK

Basic churns 4× faster than Premium

Basic customers churn at 60%, versus 14.29% for Premium — the top priority segment for intervention.

03 — REFERRAL QUALITY GAP

Referral acquisition underperforms

Referral customers churn at 83.33% versus 0% for Organic — a major difference by acquisition source.

04 — REVENUE AT RISK

Rs 73.94K in monthly revenue exposed

Churned customers represent Rs 73.94K in monthly recurring revenue — the direct financial cost of attrition.

05 — SUPPORT IS A WARNING SIGNAL

Escalations predict churn

Escalated complaints correlate with churn at 0.77, making escalation an early-warning indicator worth monitoring.

06 — GEOGRAPHIC HOTSPOTS

Three states drive most churn

Karnataka (100%), Meghalaya (66.67%) and Telangana (50%) lead churn, though small sample sizes call for caution.